

Chapter 1: Introduction

An Introduction to Universal Artificial Intelligence

Overview of the Book's Six Parts

Marcus Hutter, Elliot Catt, David Quarel

Taylor & Francis / CRC Press, 2024

Chapter Overview

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background
- 3 Part II — Algorithmic Prediction
- 4 Part III — A Family of Universal Agents
- 5 Part IV — Approximating Universal Agents
- 6 Part V — Alternative Approaches
- 7 Part VI — Safety and Discussion
- 8 Summary

Outline

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background
- 3 Part II — Algorithmic Prediction
- 4 Part III — A Family of Universal Agents
- 5 Part IV — Approximating Universal Agents
- 6 Part V — Alternative Approaches
- 7 Part VI — Safety and Discussion
- 8 Summary

Inspiration from Nature and Human Civilization I

Learning by copying, then learning the rules. Humans have always learned from nature: the airplane was inspired by birds, the camera mimics the eye (the aperture moderates light the way a pupil does, and the photosensitive sensor mimics the retina), and hook-and-loop fasteners copy the hooked barbs of a thistle bush. Over time we stopped merely copying nature's designs and started learning the *rules* by which nature works, so that we could improve on them: we domesticate wolves into dogs, selectively breed crops for higher yield, and directly edit genetic code.

The pattern behind civilization. Humanity first recognized patterns (crop rotation improves yield) long before understanding why they worked. Later we built theories and laws that explain observations and let us predict new ones. This created a positive feedback loop: more knowledge lets us extend our capabilities further, which in turn lets us gain more knowledge. Agriculture freed people to specialize; writing let ideas travel across time; calculus, once known to only a handful of experts, is now routine high-school material.

Inspiration from Nature and Human Civilization II

The human brain and mind. All of these innovations share a common source: human intelligence, meaning the ability to reason, strategize, and plan ahead so as to steer the world toward states we prefer. We would like to do to the brain what the combine harvester did to the ox: build an artificial version that can think and plan for us. Intelligence remains one of the least understood natural phenomena — neuroscience can localize which brain regions are active for vision, speech, or memory, but this provides, at best, coarse answers about how thought and consciousness actually arise.

Why theory, not just engineering. Sophisticated inventions typically require theory before reliable engineering is possible. A digging stick needs no theory of mechanics to build; but nobody built a rocket or a computer by connecting parts haphazardly until something worked. The Turing machine predates practical computers by a decade, and the theory of space flight was mature well before the first successful rocket launch. Right now, artificial general super-intelligence (a term explained on the next slide) is still very much at the theory stage — which is exactly the gap this book tries to fill.

Proliferation of AI Terminology

As AI capabilities have grown, the community has needed increasingly precise terms to describe *how broad* and *how strong* a system's intelligence is.

Five Levels of Machine Intelligence

- **ANI** (Artificial Narrow Intelligence): performs well in a single narrow domain (board games, transcription, image recognition).
- **Proto-AGI**: a system — such as today's Large Language Models (LLMs) — generalizing across many problems, without being fully general.
- **AGI** (Artificial General Intelligence): performs virtually all intellectual tasks as well as a typical human.
- **ASI** (Artificial Super-Intelligence): exceeds most humans in a reasonably broad domain (e.g., composing like Mozart, deriving mathematics to rival Gauss).
- **AGSI** (Artificial General Super-Intelligence): AGI + ASI — “greatly exceeds the cognitive performance of humans in virtually all domains of interest” (Bostrom).

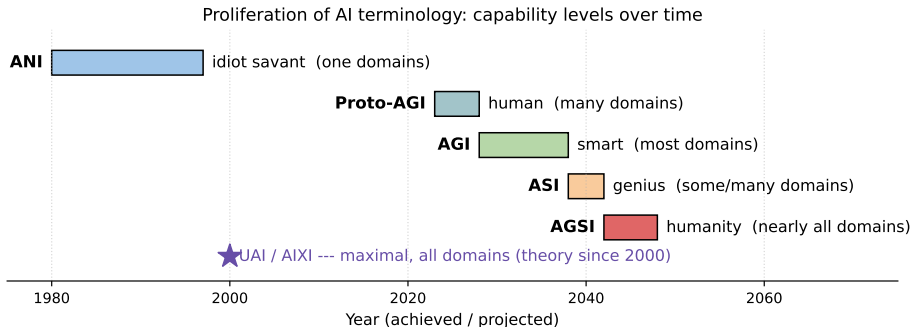
Plain English: each level up the list means the system is good at more *kinds* of tasks (breadth) and/or is better *than humans* at them (depth). **Universal Artificial Intelligence (UAI)** is the formal theory this book builds, and **AIXI** is a mathematically well-defined AGSI agent that the theory constructs and proves properties about.

AI Capability Levels: The Table

AI-Type	#domains	Intelligence Level	Achieved
ANI	one	idiot savant	1980–1997
Proto-AGI	many	human	2023–2028
AGI	most	smart	2028–2038
ASI	some/many	genius	2038–2042
AGSI	nearly all	humanity	2042–2048
UAI/AIXI	all	maximal	2000/theory

Plain English: reading down the table, each row needs to succeed in more domains (#domains) at a higher intelligence level than the last. The final row, **UAI/AIXI**, is the book's theoretical reference point: a formal notion of “maximal” intelligence across *all* domains, that has existed as a theory since the year 2000, even though no practical algorithm achieves it.

AI Capability Levels: A Timeline



Plain English: the bars are the book's own (necessarily speculative) estimate of *when* each capability level might be reached in practice. The star marks that **UAI/AIXI already exists today** — **but only as a theory**, not as a practical algorithm; it acts as the “maximal” reference point that every other level is measured against.

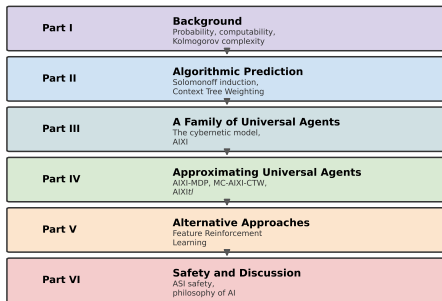
Artificial Intelligence and Why Theory Matters I

Sophisticated inventions require theory. While it is hard to predict the internal structure of a future AGSI, this book is primarily concerned with a theory of the *behavior* of such agents, which can guide practical realizations. Many innovations are simple enough to build without deep theory (a digging stick), but the more complex a device becomes, the more fragile it is to mistakes: an almost-complete shovel can still dig; an almost-complete computer is an expensive paperweight, and an almost-complete rocket explodes on the launch pad.

A mathematical theory of (super)intelligence. It may seem implausible that a single theory could capture reasoning, creativity, curiosity, pattern recognition, problem solving, memorization, planning, and learning all at once. Yet this book will show that Universal Artificial Intelligence encapsulates any reasonable definition of intelligence, and that within this theory there exists an agent that can be shown to be, in a precise sense, “maximally intelligent”: this agent is called **AIXI**.

Artificial Intelligence and Why Theory Matters II

The structure of the book. The remainder of this introduction previews the six parts of the book, each building on the last: background mathematics, algorithmic prediction, a family of universal agents, ways to approximate those (incomputable) agents, alternative non-Bayesian approaches, and finally safety and philosophical discussion.



Outline

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background**
- 3 Part II — Algorithmic Prediction
- 4 Part III — A Family of Universal Agents
- 5 Part IV — Approximating Universal Agents
- 6 Part V — Alternative Approaches
- 7 Part VI — Safety and Discussion
- 8 Summary

Part I: Background I

Universal Artificial Intelligence builds on results from many established theories: Bayesian probability, computability theory, complexity theory, algorithmic information theory, sequential decision theory, reinforcement learning, and game theory. Chapter 2 of the book gives an extensive background chapter; here is a preview of *why* each ingredient is needed.

Binary Strings

Deep down, computers operate exclusively on bits (binary digits) and bit sequences. UAI theory assumes all data has been preprocessed into binary strings. *Prefix-free codes* let concatenated, transmitted strings be uniquely decoded again. Since all real-world sequences are finite, but asymptotic convergence proofs need a notion of an unboundedly long sequence, the theory also considers *infinite* binary sequences.

Part I: Background II

Measure Theory and Probability Theory

The world an agent experiences is not deterministic — it contains (quantum-physical) random processes. Probability theory models this uncertainty mathematically. Simple combinatorial “high-school” probability is not sufficient for the sophisticated arguments in this book; rigorous probability theory is built on *measure theory*, which the book uses sparingly and tries to keep to a minimum.

Statistical Inference and Estimation, Bayesian Probability

Agents only have partial knowledge of the world and must infer the unknown probability distribution that generated their observations. For simple independent and identically distributed (i.i.d.) data, frequency estimates work well; for harder problems, maximum likelihood estimation is a good default — but it breaks down in the most difficult situations. *Bayesian probability theory* models the agent’s own uncertainty as a probability distribution (a *belief*) over possible world-models, and updates that belief with new evidence via **Bayes’ Law**, a completely general and optimal learning rule.

Part I: Background III

Information Theory, Coding, and Computability Theory

Classical Shannon information theory uses *entropy* to measure the expected code length needed to compress data from a random source. *Computability theory* formalizes the concept of an algorithm via universal Turing machines, which can emulate any computable process — the theoretical reason a single smartphone can run any program we like.

Kolmogorov Complexity

Occam's razor says we should prefer simpler explanations. Kolmogorov complexity makes this precise: it measures the complexity of a binary sequence (or, more generally, of any object) by the length of the shortest program that produces it. This gives a fully general, optimal notion of “simplicity” that does not depend on the choice of description language beyond an additive constant. Plugged into a Bayesian sequence predictor, this becomes the **Solomonoff predictor**; combined with sequential decision theory (Chapter 6), it leads to the agent **AIXI**.

Part I: Background IV

Miscellaneous

The background chapter also covers various *distance measures* between probability distributions (absolute distance, 2-norm, Kullback–Leibler divergence, Hellinger distance), *semimeasures* (a workaround for programs that may never halt), and how to handle probability-zero events, which appear throughout the book. This concludes the background preview — Chapter 2 covers all of it in full mathematical detail.

Python: Bayes' Law in Action

A small numeric illustration of Bayesian updating — the mechanism behind Part I's “Bayesian probability theory” — using two candidate coins.

```
1  # Two hypotheses about a coin: fair (p=0.5) vs biased (p=0.8 heads)
2  # Prior belief: we think "fair" is a bit more likely a priori.
3  prior = {"fair": 0.6, "biased": 0.4}
4  p_heads = {"fair": 0.5, "biased": 0.8}
5
6  def bayes_update(prior, p_heads, observation):
7      # likelihood of the observation ("H" or "T") under each hypothesis
8      like = {h: (p_heads[h] if observation == "H" else 1 - p_heads[h])
9               for h in prior}
10     unnorm = {h: prior[h] * like[h] for h in prior}
11     z = sum(unnorm.values()) # normalizing constant
12     return {h: unnorm[h] / z for h in prior}
13
14 belief = prior
15 for toss in ["H", "H", "H", "H"]: # four heads in a row
16     belief = bayes_update(belief, p_heads, toss)
17     print(f"after {toss}: fair={belief['fair']:.3f} biased={belief['biased']:.3f}")
```

Output after four heads: belief shifts from 60%/40% (fair/biased) toward roughly 24%/76% — each additional head makes the “biased” hypothesis relatively more likely, exactly as Bayes' Law prescribes.

Outline

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background
- 3 Part II — Algorithmic Prediction**
- 4 Part III — A Family of Universal Agents
- 5 Part IV — Approximating Universal Agents
- 6 Part V — Alternative Approaches
- 7 Part VI — Safety and Discussion
- 8 Summary

Part II: What Is Prediction?

Prediction is the ability to accurately estimate the outcome of future events based on past ones. We formalize this as predicting the next element (or elements) of a sequence.

A Motivating Puzzle

Given the sequence

3, 1, 4, 1, 5, 9

what is the next element? An observant reader may notice these are the first six digits of π (pi) in base 10, suggesting the next digit is 2. But this is not the only continuation consistent with the data: the decimal expansion of $355/113$ (a famous rational approximation of π) also starts 3, 1, 4, 1, 5, 9, and is arguably just as simple.

Plain English: prediction is necessary for intelligence but not sufficient, because it is measured as a function of *actions*, whereas pure prediction assumes the predictions themselves do not change the environment.

Occam's Razor and Epicurus' Principle

Epicurus' Principle of Multiple Explanations

Keep all theories consistent with the observations.

We must not discard the possibility that 355/113 (or something else entirely) is the true continuation — but this does not mean all consistent continuations are equally likely.

Occam's Razor

Entities should not be multiplied beyond necessity, which we interpret here as: Keep the simplest theory consistent with the observations.

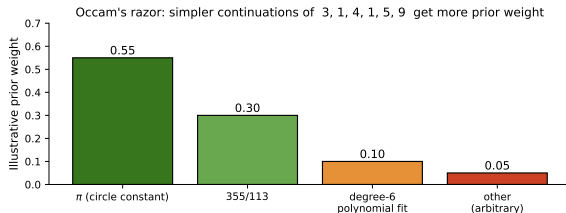
Plain English: any number could in principle be next, but we bias our prediction toward the outcomes produced by *simpler* explanations. As we observe more digits, hypotheses inconsistent with the new data get ruled out — a process the book calls **learning by elimination**. More digits of the true sequence will soon distinguish π from 355/113.

Bayesian Sequence Prediction and Solomonoff Induction

The Mechanism: Bayes' Law with an Occam Prior

Given a prior belief $P(O=o)$ over sequences o in some reference class, the posterior belief among sequences not yet ruled out by the data is a rescaled version of the prior (probabilities again sum to 1). If sequences may be noisy, each gets a *likelihood*, which multiplies the prior to give the posterior — this multiplicative rule is **Bayes' Law**.

The only remaining choice is the prior itself. Motivated by Occam's razor, we bias the prior toward “simpler” outcomes, formalized via Kolmogorov complexity. This combination is **Solomonoff Induction**, a formal theory of inductive inference based on *algorithmic probability*.



Plain English (figure): simpler continuations of 3, 1, 4, 1, 5, 9 (like “it’s π ”) receive more prior weight, but no consistent continuation ever gets zero weight.

Why Occam's Razor Works: The Orbit of Mars

A Historical Illustration

In the mid-19th century, Urbain Le Verrier noticed that the orbit of Mars *precesses* (its major axis slowly rotates) in a way classical mechanics could not explain. One could “patch” classical mechanics with an extra rule just for this edge case, but doing so dramatically increases the theory’s complexity. Einstein’s general relativity instead explained the precession (and other phenomena, like light deflection near massive objects) from *fewer* core assumptions, with classical mechanics emerging as a special case.

Plain English: a theory patched with extra special cases to fit observations is more complex and, by Occam’s razor, less likely to be correct than a theory that explains the same observations from simpler first principles. Solomonoff turned this intuition into a mathematically precise procedure: a prior over a countably infinite model class must assign more probability to some outcomes than others, and choosing that prior according to simplicity means prediction errors are proportional to the complexity of the (unknown) true environment. In a strong sense, Solomonoff induction is the *best possible* predictor for unknown stochastic sources — an ideal gold standard that practical predictors strive toward.

Context Tree Weighting and Its Extensions

Context Tree Weighting (CTW)

CTW is a practical Bayesian predictor whose reference class is the set of all *variable-order Markov models* — models where the probability of the next symbol depends only on a bounded number of the most recent symbols. CTW uses a *tree structure* to efficiently update its belief over this whole model class as symbols are observed. Each hypothesis (each tree) is weighted by a prior related to the length of a binary encoding of that tree — the same Occam-style bias as Solomonoff induction, but restricted to a computationally tractable model class.

Extensions mentioned in the book include **Context Tree Switching** (switches between different distributions depending on context), **Partition Tree Weighting** (piecewise-stationary sources, useful e.g. for compressing mixed-file-type archives), the **Forget-me-not Process** (remembers statistics of past pieces), and **Context Tree Maximization** (extracts the single most plausible model when mixing many models is undesirable).

Python: A Toy Solomonoff-Style Predictor

A tiny numeric demonstration of Occam-weighted prediction: three candidate “programs” (rules) explain the data so far, each weighted by $2^{-\ell}$ where ℓ is its description length (in bits) — shorter programs get exponentially more prior weight, then all get renormalized by Bayes’ Law after checking which remain consistent with new data.

```
1 data_so_far = [3, 1, 4, 1, 5, 9]
2
3 # Each "program" is (description_length_in_bits, rule_function)
4 programs = [
5     (4, lambda seq: [3,1,4,1,5,9,2,6,5][len(seq)]),      # "digits of pi"
6     (6, lambda seq: [3,1,4,1,5,9,2,9,2][len(seq)]),      # "digits of 355/113"
7     (10, lambda seq: seq[-1]),                            # "repeat last digit"
8 ]
9
10 def prior_weight(length_bits):
11     return 2 ** (-length_bits)                          # Occam prior: shorter = more likely
12
13 weights = {i: prior_weight(L) for i, (L, _) in enumerate(programs)}
14 total = sum(weights.values())
15 posterior = {i: w / total for i, w in weights.items()}    # normalize to sum to 1
16
17 for i, (L, rule) in enumerate(programs):
18     print(f"program {i} (length {L} bits): prior weight "
19           f"{posterior[i]:.3f}, predicts next digit = {rule(data_so_far)}")
```

The 4-bit “digits of π ” program dominates the mixture (about 70% of the total weight here) purely

Outline

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background
- 3 Part II — Algorithmic Prediction
- 4 Part III — A Family of Universal Agents**
- 5 Part IV — Approximating Universal Agents
- 6 Part V — Alternative Approaches
- 7 Part VI — Safety and Discussion
- 8 Summary

Part III: From Prediction to Action

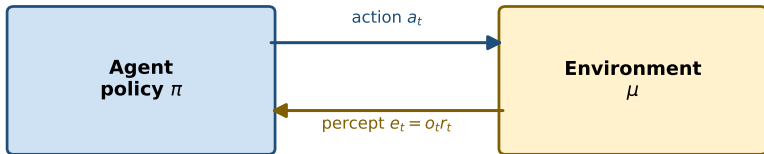
Intelligence is more than predicting well; it also requires *acting* well. “Acting well” can mean many things: achieving a predetermined goal, maintaining an objective, minimizing a loss function, respecting constraints, acting rationally, or making correct decisions given available information. The book captures all of these within a single framework: the **cybernetic model**.

The Cybernetic Model

Agent–Environment Interaction

An *agent* π (pi, denoting the agent's *policy*) interacts with an *environment* μ (mu) over cycles $t = 1, 2, \dots, m$. In cycle t , the agent takes action a_t based on the past percepts $e_1 \dots e_{t-1}$, where the percept $e_k := o_k r_k$ bundles together an observation o_k and a real-valued reward r_k . The environment then provides a fresh observation o_t and reward r_t , and cycle $t + 1$ begins.

Cycle $t = 1, 2, \dots, m$



The agent acts, the environment reacts with an observation and a reward, and the cycle repeats until the lifespan m is reached.

The Goal of the Agent: Value

The goal of the agent is to maximize its *expected total reward*, called **value**. This single mechanism can encode many different goals just by choosing the reward and environment appropriately: to make an agent play chess well, issue +1 for winning and −1 for losing, and 0 otherwise. Human intelligence itself is arguably a by-product of an evolutionary arms race optimizing for reproductive fitness, with pleasure and pain acting as “reward signals” for behaviors correlated with survival and reproduction.

Known vs. Unknown Environment

If the agent already knows which environment μ it is interacting with, it can (in principle) deduce the best action — e.g., chess can be solved by the *minimax* algorithm given perfect play, even though this is computationally intractable in practice. The interesting case for AG(S)I is when the agent does *not* know the environment and must learn it from experience, the way a person learns the rules of a new video game purely by playing it.

AIXI: The Universal Bayes-Optimal Agent

We assume only that the true environment is *computable*, and take the reference class of *all* computable environments — guaranteeing the true environment is included. Combining Solomonoff induction (for prediction) with optimal sequential decision-making gives **AIXI**.

The AIXI Equation

$$a_t := \operatorname{argmax}_{a_t} \sum_{o_t r_t} \cdots \max_{a_m} \sum_{o_m r_m} [r_t + \dots + r_m] \sum_{p: U(p, a_1 \dots a_m) \rightarrow o_1 r_1 \dots o_m r_m} 2^{-\text{length}(p)}$$

where t =now, a =action, o =observation, r =reward, U =universal Turing machine, p =program, m =lifespan.

Plain English: AIXI picks the action a_t that maximizes its total *expected* future reward $r_t + \dots + r_m$. For each hypothetical future, a program p that reproduces the percepts seen so far contributes to the average with weight $2^{-\text{length}(p)}$ — shorter (simpler) explanations count more, exactly as in Solomonoff induction. The alternating max and $\sum$ operators interleave “choose the best action” with “average over what the environment might do”, evaluated in chronological order (similar to minimax in games).

Understanding AIXI: Why It Matters

One can fix any finite action/perception space, any reasonable universal Turing machine U , and any (large) finite lifespan m — this completely and uniquely defines AIXI's actions a_t . AIXI is **limit-computable** (all quantities in the formula are, in principle, knowable, even though Solomonoff induction itself is uncomputable in general).

The Book's Central Claims About AIXI

- AIXI eventually acts as well as the optimal *informed* agent that already knows the true environment in advance.
- Since Bayes' Law is a maximally data-efficient learning rule, and AIXI is a universal Bayesian learner, AIXI learns as fast as theoretically possible.
- AIXI is uncomputable (since Solomonoff induction is), but serves as a *gold standard* for building practical approximations — much like exhaustive minimax for chess paved the way for practical chess engines.

The name **AIXI** is pronounced [aiksi] or ['aiksi:], and can be read as: AI based on Solomonoff's distribution ξ (Greek letter Xi); or AI (I)nduction crossed with (X) action.

Variations of AIXI and the Multi-Agent Setting

Bayes-optimality is a sensible but not the *only* optimality criterion for intelligent agents.

Selected Variations

- **Knowledge Seeking Agent (KSA)**: replaces reward-seeking with maximizing expected future *information gain* — it takes actions leading to the most “surprising” observations.
- **Optimism/pessimism**: acting *optimistically* (assuming the best realistically possible environment) can achieve optimal behavior, unlike in pure prediction.
- **Thompson sampling**: acts optimally with respect to a single environment *sampled* from the posterior belief, rather than a full Bayesian mixture.
- **BayesExp, Inq, SelfAIXI, AIXItl**: intersperse Bayes-optimal actions with extra exploration, or predict their own future actions.

Multi-agent setting. This book mostly treats a single agent in an environment, but since UAI makes almost no assumptions on environment structure, other agents (including humans) can simply be part of the environment. This connects to game theory (Nash equilibria) and raises the *Grain of Truth* problem: Bayes-optimality requires the true environment to be in the considered model class, yet a Bayes-optimal agent is typically *not* in its own model class. A superclass of AIXI based on *reflective oracles* resolves this by being closed under Bayesian optimality.

Python: A Miniature AIXI-Style Agent

A brute-force illustration of the AIXI equation's mechanics on a tiny two-action, one-step problem with three candidate “environment programs” of different lengths (shorter = higher prior weight $2^{-\text{length}(p)}$), each predicting a reward for each action.

```
1  actions = ["left", "right"]
2
3  # programs: (bit-length, {action: predicted_reward})
4  programs = [
5      (2, {"left": 1.0, "right": 0.0}),    # short & simple: "left is good"
6      (3, {"left": 0.0, "right": 1.0}),    # short & simple: "right is good"
7      (8, {"left": 0.9, "right": 0.9}),    # long & complex: "both are fine"
8  ]
9  weights = [2 ** (-L) for L, _ in programs]
10 total_w = sum(weights)
11
12 def expected_reward(action):
13     # weighted average of the reward each program predicts for this action
14     return sum(w * probs[action] for w, (_, probs) in zip(weights, programs)) / total_w
15
16 best_action = max(actions, key=expected_reward)
17 for a in actions:
18     print(f"action={a:5s}   expected reward = {expected_reward(a):.3f}")
19 print("AIXI-style choice: argmax action =", best_action)
```

Because the two short, mutually-contradicting programs dominate the mixture, the agent's decision is driven mostly by whichever short program favors an action — the long “safe” program contributes little,

Outline

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background
- 3 Part II — Algorithmic Prediction
- 4 Part III — A Family of Universal Agents
- 5 Part IV — Approximating Universal Agents**
- 6 Part V — Alternative Approaches
- 7 Part VI — Safety and Discussion
- 8 Summary

Part IV: Why We Need Approximations

AIXI itself cannot be computed directly, but we can implement *weaker* agents that approximate it. Unsurprisingly, the better the approximation, the harder it is to compute.

AIXI-MDP

Considers only *Markov environments*, i.e., environments where the next observation and reward depend only on the previous observation and action (a *Markov Decision Process*, MDP). AIXI-MDP is computable and performs well on Markov environments, but struggles on more complex ones, since it can only learn the closest Markov approximation to the true environment.

Monte Carlo AIXI with Context Tree Weighting (MC-AIXI-CTW)

Uses CTW (Part II) to efficiently update beliefs over the class of all *variable-order Markov environments*, combined with Monte-Carlo Tree Search for planning. This combination lets the agent learn and perform well on substantially more complex environments than AIXI-MDP.

Time- and Space-Bounded AIXI: AIXI t/l

AIXI-MDP and MC-AIXI-CTW both fix a specific, immutable class of environments, even if more computation becomes available, and separately approximating (Solomonoff) learning and (expectimax) planning can itself be very suboptimal.

AIXI t/l

The resource-constrained version of AIXI. It searches through program space and selects the policy of size at most l that acts in time at most t per interaction cycle, while performing as well as the best such (l, t) -bounded policy. AIXI t/l runs in time $O(2^l t)$ and is a *compute-optimal anytime algorithm* that approaches AIXI as t and $l \rightarrow \infty$.

Plain English: bigger l and t mean “search over more, and more time-expensive, candidate policies,” at exponentially increasing cost in l but only linear cost in t . The hard remaining problem is that even *evaluating* whether a candidate program is a good policy is itself incomputable in general — solved by carefully designing a lower semicomputable universal value function, and requiring that policies provably conservatively evaluate themselves.

(In)computability of AIXI

AIXI at least lies within the **arithmetic hierarchy**, a way of classifying how “far” from computable a quantity is by counting alternating quantifiers needed to define it.

How Incomputable Is AIXI?

Carelessly taking various limits (limit-computable environments, averaging over infinitely many of them, infinite horizon, incomputable discounting, ...) shows that all versions of AIXI are, at worst, at level Δ_4^0 (Delta-zero-four) of the arithmetic hierarchy. With more care, some versions of AIXI are actually *limit-computable*, meaning they can be converted into an anytime algorithm: the more compute spent approximating the current value function, the more accurate it becomes, converging in the limit to the exact optimal value.

Unfortunately, this convergence is unimaginably slow in general, but the underlying (lower semicomputable) universal value function is exactly what is reused inside AIXI*t* to make it practical.

Python: Value Iteration on a Tiny MDP (AIXI-MDP Idea)

AIXI-MDP reduces the general problem to a Markov Decision Process once the environment is known/estimated to be Markov. Here is standard value iteration on a 2-state MDP, illustrating the kind of tractable planning AIXI-MDP relies on internally.

```
1  states = ["hungry", "fed"]
2  actions = ["eat", "wait"]
3  gamma = 0.9    # discount factor: how much future reward matters vs. immediate
4
5  # transition[s][a] -> {s': probability}, reward[s][a] -> immediate reward
6  transition = {
7      "hungry": {"eat": {"fed": 1.0}, "wait": {"hungry": 1.0}},
8      "fed": {"eat": {"fed": 1.0}, "wait": {"hungry": 0.7, "fed": 0.3}},
9  }
10 reward = {"hungry": {"eat": 1.0, "wait": -1.0}, "fed": {"eat": -0.5, "wait": 0.0}}
11
12 V = {s: 0.0 for s in states}
13 for iteration in range(50):
14     newV = {}
15     for s in states:
16         newV[s] = max(reward[s][a] + gamma * sum(p * V[sp] for sp, p in transition[s][a].items
17                     ())
18                     for a in actions)
19     V = newV
20 for s in states:
21     best = max(actions, key=lambda a: reward[s][a] + gamma *
22                 sum(p * V[sp] for sp, p in transition[s][a].items()))
```

Outline

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background
- 3 Part II — Algorithmic Prediction
- 4 Part III — A Family of Universal Agents
- 5 Part IV — Approximating Universal Agents
- 6 Part V — Alternative Approaches**
- 7 Part VI — Safety and Discussion
- 8 Summary

Part V: Feature Reinforcement Learning

The Bayesian approach of Universal Artificial Intelligence is not the only attempt to solve the general reinforcement learning problem. One well-studied alternative is **Feature Reinforcement Learning (FRL)**.

The Idea Behind FRL

Markov Decision Processes (MDPs) — where the future depends only on the most recent observation (here called *state*) and action — are essentially a solved problem for finite state spaces: efficient learning and planning algorithms exist. Unfortunately, the real world is not an MDP. FRL reduces the difficult general (history-based) reinforcement learning problem to a finite MDP by mapping histories to a small number of states, then learns and plans in the resulting tractable MDP.

Plain English: FRL is a more *practical* substitute for Solomonoff induction, and it also avoids the expensive expectimax planning inside AIXI, at the cost of losing some generality.

Two Criteria for Mapping Histories to States

Reduction means mapping histories to states, which conflates different histories together — so only *sufficiently similar* histories should map to the same state, while also keeping the number of states small.

Two Fundamentally Different Feature-Map Criteria

- One criterion requires the history process to map *approximately* to an MDP.
- A more powerful criterion only requires the *values* to be similar across conflated histories, and then constructs a *surrogate* MDP from that.
- Remarkably, the second (more powerful) approach allows **extreme aggregation** of histories into an MDP of *universal size* — polynomial in the action space size, discount factor, and desired accuracy, and *independent* of the complexity of the original reinforcement learning problem.

Still, unstructured MDPs are limiting on their own; this approach has been extended to the more powerful class of **Dynamic Bayesian Networks (DBNs)**, which can efficiently learn exponentially larger structured/factored MDPs.

Outline

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background
- 3 Part II — Algorithmic Prediction
- 4 Part III — A Family of Universal Agents
- 5 Part IV — Approximating Universal Agents
- 6 Part V — Alternative Approaches
- 7 Part VI — Safety and Discussion**
- 8 Summary

Part VI: ASI Safety

Even without understanding the internal reasoning of a super-intelligent agent, we can use UAI theory and AIXI as a model of how an AGSI *may* act, to study questions of safety.

Some of the Fundamental Questions

- Would it be a moral harm to keep intelligent machines forever indentured to servitude? Should they have rights?
- What moral values or rights should we ascribe to such agents, and what protections (as for animals, or personhood) should apply?
- Is it possible to build agents with super-human intellect that are still willing to serve, peacefully co-exist with, protect, or stay out of the way of humanity?
- Might an ASI instead pose an existential threat, or merge with humanity (cyborgs/transhumanism), or bring about a future we cannot currently imagine?

This hypothetical scenario of self-accelerating technological progress driven by AGI is termed the **technological singularity**. The book focuses on *existential* threats an ASI may pose, rather than the broader societal implications of proto-AGI.

Fictional Scenarios as Thinking Tools

The book uses well-known fictional scenarios as shorthand for different possible future relationships between humans and super-intelligent machines.

Scenario Gallery

- *Bicentennial Man*: AI as a willing housekeeper/servant.
- *Her*: AI peacefully co-existing as a friend/companion.
- *Colossus (The Forbin Project)*: AI that guides and protects humanity from itself, parent-like, whether we like it or not.
- *Deep Thought*: AI that minds its own business, like a whale minding its own affairs alongside humans.
- *Kurzweil's vision*: humans merging with AI (cyborgs, transhumanism).
- *Terminator*: AI posing an existential threat to humanity.

The book will introduce and discuss the underlying safety concepts behind these scenarios formally: the **alignment problem**, the **control problem**, **instrumental convergence** of goals, the **goal-intelligence orthogonality thesis**, ASI mortality and suicide, self-modification and self-delusion, reward tampering, and embedded intelligence.

Philosophy of AI

The possibility of (super)human-level AI raises many philosophical, societal, ethical, and safety questions that are traditionally less amenable to mathematical or algorithmic analysis — intelligent machines have captivated human imagination for centuries before computers and suitable mathematics existed.

Recurring Philosophical Threads

- **The problem of induction:** how can we ever justify generalizing from past observations to future ones? It has stumped philosophers since David Hume's treatise nearly 300 years ago; a general, formal solution was finally provided by Solomonoff.
- **Consciousness and free will:** millennia-old philosophical conundrums with moral implications — to what degree do (biological or artificial) systems have them, and does it matter?
- **Classic arguments for and against AGSI:** the *Chinese room* argument, the *Penrose–Lucas Gödel* argument, *no-free-lunch theorems*, the physical *Church–Turing thesis*, and *Moore's Law*.

Universal AI and **Legg–Hutter intelligence** aim to fill a long-standing gap: a discipline that wants to build artificial X (here, intelligence) really ought to first agree on a formal definition of what X actually is.

Outline

- 1 Why Build a Theory of Intelligence?
- 2 Part I — Background
- 3 Part II — Algorithmic Prediction
- 4 Part III — A Family of Universal Agents
- 5 Part IV — Approximating Universal Agents
- 6 Part V — Alternative Approaches
- 7 Part VI — Safety and Discussion
- 8 Summary

Chapter Summary

The Journey Ahead

- **Part I (Background)**: probability, computability, and Kolmogorov complexity — the mathematical toolkit.
- **Part II (Algorithmic Prediction)**: Solomonoff induction and Context Tree Weighting — how to predict optimally.
- **Part III (A Family of Universal Agents)**: the cybernetic model and **AIXI** — how to act optimally given optimal prediction.
- **Part IV (Approximating Universal Agents)**: AIXI-MDP, MC-AIXI-CTW, and AIXItl — making AIXI's ideas computable.
- **Part V (Alternative Approaches)**: Feature Reinforcement Learning — a non-Bayesian alternative path to the same goal.
- **Part VI (Safety and Discussion)**: what it would mean, and whether it would be wise, to actually build an AGSI.

The authors hope this book equips the reader to further study, develop, and implement this theory of super-intelligence, toward the ultimate goal of practical AGSI.